

软件过程改进方案设计文档

一、项目概述

1.1 项目背景

本项目为校园闲置物品交易小程序，主要服务于高校学生群体，为用户提供闲置商品发布、商品浏览、在线沟通、订单管理以及校园自提预约等功能。

原软件过程采用“瀑布模型 + 敏捷开发思想”的轻量化开发模式，共包含：

- 需求分析与规划
- 原型设计与架构设计
- 开发实现
- 单元测试与集成测试
- 部署与上线
- 运维与迭代

随着生成式 AI、大语言模型（LLM）、智能体（Agent）等技术的快速发展，传统软件开发流程正在向“人机协同开发模式”转变。

因此，本方案拟在保留原过程核心结构的基础上，引入 AI 能力，对软件生命周期进行智能化升级。

二、改进目标

本次过程改进主要目标如下：

2.1 提高开发效率

通过 AI 自动生成需求文档、设计文档、代码及测试用例，减少重复性劳动。

目标：

- 开发效率提升 40%以上
- 文档编写时间减少 60%以上

2.2 提高软件质量

利用 AI 进行需求检查、代码审查和测试覆盖分析。

目标：

- 缺陷率降低 30%
 - 测试覆盖率提高至 90%以上
-

2.3 降低单人开发成本

项目为个人开发项目。

引入 AI 后：

AI 承担部分产品经理、架构师、测试工程师职责。

开发者专注核心业务实现。

三、AI 增强软件过程总体设计

3.1 AI 增强软件过程框架

flowchart TD

A[需求调研]

-->B[AI 需求分析]

B

-->C[需求评审]

C

-->D[AI 架构设计]

D

-->E[原型设计]

E

-->F[AI 辅助开发]

F

-->G[代码审查]

G

-->H[AI 测试生成]

H

-->I[自动化测试]

I

-->J[部署上线]

J

-->K[AI 运维监控]

K

-->L[版本迭代]

3.2 过程特点

与传统过程相比：

新增：

- AI 需求分析节点
- AI 设计生成节点
- AI 代码生成节点
- AI 测试生成节点
- AI 运维分析节点

形成：

“人工决策 + AI 执行”的协同模式。

四、各阶段改进方案

4.1 需求分析阶段改进

原过程

开发者：

- 用户调研
- 编写需求文档
- 制定开发计划

存在问题：

- 耗时较长
 - 易遗漏需求
 - 需求文档格式不统一
-

AI 改进方案

采用：

- ChatGPT
- Claude
- Gemini

辅助需求分析。

输入

开发一个校园二手交易平台

支持：

商品发布

商品搜索

在线聊天

订单管理

校园自提

AI 自动生成

输出：

- 用户画像
 - 用户故事
 - 功能需求
 - 非功能需求
 - 验收标准
-

新增活动

AI 需求生成

生成：

《AI 需求草案》

AI 需求一致性检查

检查：

- 冲突需求
 - 重复需求
 - 缺失需求
-

新交付物

交付物

AI 需求分析报告

交付物

用户故事清单

需求一致性检查报告

4.2 系统设计阶段改进

原过程

人工完成：

- 原型设计
 - 架构设计
 - 数据库设计
-

AI 改进方案

采用：

- ChatGPT
- Mermaid
- PlantUML

辅助设计。

AI 自动生成架构

输入：

设计校园闲置交易平台

生成：

- 系统架构图
 - 模块划分图
-

AI 自动生成数据库设计

自动输出：

- 用户表
- 商品表
- 订单表
- 聊天记录表

及其关联关系。

AI 自动生成 UML

生成：

- 类图
 - 时序图
 - 用例图
-

新交付物

交付物

AI 架构设计方案

AI 数据库设计报告

AI UML 模型

4.3 开发实现阶段改进

原过程

开发者独立编码。

存在：

- 重复代码较多
 - CRUD 开发耗时
-

AI 改进方案

采用：

- Cursor
 - GitHub Copilot
 - Claude Code
-

自动生成代码

AI 自动生成：

- Controller
 - Service
 - Mapper
 - DTO
 - VO
-

自动生成接口文档

自动输出：

接口地址

请求参数

返回参数

示例数据

AI 代码重构

自动发现：

- 重复代码
- 冗余逻辑
- 性能问题

并给出优化建议。

新交付物

交付物

AI 生成代码记录

代码审查报告

重构建议报告

4.4 测试阶段改进

原过程

人工设计测试用例。

问题：

- 覆盖率有限
 - 易遗漏异常场景
-

AI 改进方案

采用：

- ChatGPT
 - Diffblue
 - Copilot
-

自动生成测试用例

覆盖：

功能测试

例如：

正常登录

异常测试

密码错误

边界测试

空值

超长字符

安全测试

SQL 注入

XSS 攻击

自动生成 JUnit

自动生成:

@Test

```
public void loginTest()
```

新交付物

交付物

AI 测试用例库

自动化测试脚本

覆盖率分析报告

4.5 运维阶段改进

原过程

人工分析日志。

AI 改进方案

采用:

- ELK + AI

- Datadog AI
 - Azure OpenAI
-

AI 日志分析

自动识别:

- 慢查询
 - 500 错误
 - CPU 异常
-

根因分析

自动判断:

数据库连接池耗尽

Redis 异常

网络超时

智能优化建议

生成:

- SQL 优化方案
 - 缓存优化方案
 - 服务器扩容建议
-

新交付物

交付物

异常分析报告

性能优化建议

AI 运维周报

五、新增角色设计

5.1 AI 过程工程师

职责：

- 管理 AI 工具
- 编写 Prompt 模板
- 监督 AI 输出质量
- 建立知识库

5.2 AI 测试分析师

职责：

- 审核 AI 测试用例
- 分析测试覆盖率
- 优化测试策略

5.3 开发者

职责调整：

由传统执行者转变为：

- AI 结果审核者
- 核心业务决策者
- 架构优化者

六、工具链选型

阶段	工具	用途
需求分析	ChatGPT	需求生成
需求评审	Claude	需求检查

阶段	工具	用途
系统设计	Mermaid AI	架构设计
编码实现	Cursor	代码生成
代码审查	CodeRabbit	Review
测试验证	Diffblue	测试生成
运维监控	Datadog AI	日志分析

七、效率提升评估

活动	传统耗时	AI 耗时	提升比例
需求分析	4h	1h	75%
系统设计	8h	3h	62.5%
编码开发	40h	24h	40%
测试设计	3h	0.5h	83%
故障定位	1h	15min	75%

预计整体开发效率提升：

40% ~ 60%

八、风险分析与应对策略

风险一：AI 幻觉

问题：

生成错误代码。

措施：

- 人工 Review
- 单元测试验证

风险二：数据安全风险

问题：

敏感代码上传云端。

措施：

- 私有化部署模型
 - 数据脱敏
-

风险三：技术依赖

问题：

开发者能力退化。

措施：

- 定期代码 Review
 - 技术培训
-

风险四：生成结果质量不稳定

问题：

不同 Prompt 结果不同。

措施：

- 建立标准 Prompt 模板库
 - 输出结果双重审核
-

九、总结

本方案在保留原软件过程主体结构的基础上，引入 AI 需求分析、AI 架构设计、AI 代码生成、AI 测试生成和 AI 运维分析五大能力，实现软件生命周期的全面智能化升级。

改进后的过程能够有效降低开发成本，提高软件质量，提升项目交付效率，为未来智能软件工程实践提供参考。